



第6课-Triton基础-复现大模型中的mlp算子

配套视频

[第6课-复现大模型结构中的mlp层.mkv](#)

前言

在前面的章节中，我们已经学习了如何使用Triton编写矩阵乘法算子，包括在Deepseek模型中使用的FP8量化矩阵乘法算子(fp8_gemm)，以及一些基础的线性算子，如Softmax和RMSNorm等。今天，我们将进一步探讨如何用Triton实现一个完整的MLP（多层感知器）层。让我们首先来分析LLama3中MLP层的实现方式。为了便于理解，我对原始代码进行了适当简化。

llama中MLP层的Pytorch实现

```
1  class FeedForward(nn.Module):
2      def __init__(
3          self,
4          dim: int,
5          hidden_dim: int,
6          multiple_of: int,
7      ):
8          super().__init__()
9          hidden_dim = int(2 * hidden_dim / 3)
10         hidden_dim = multiple_of * ((hidden_dim + multiple_of - 1) // multiple_of)
11
12         self.w1 = nn.Linear(
13             dim, hidden_dim, bias=False
14         )
15         self.w2 = nn.Linear(
16             hidden_dim, dim, bias=False
17         )
18         self.w3 = nn.Linear(
19             dim, hidden_dim, bias=False
20         )
21
22     def forward(self, x):
23         w1x = self.w1(x)
24         w3x = self.w3(x)
25         silu_w1x = F.silu(w1x)
```

我们将以上述FeedForward函数作为LLaMA中MLP层的基准实现，并用它来验证我们的Triton实现的正确性。从代码分析可知，我们需要实现两个关键组件：SiLU激活函数和逐元素乘法运算。

SiLU激活函数的数学表达式为：

- $\text{SiLU}(x) = x \cdot \text{sigmoid}(x)$
- 其中 $\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$
- 因此完整形式为： $\text{SiLU}(x) = \frac{x}{1 + e^{-x}}$

另外的Mul层是将两个输入的张量每个位置的元素逐点相乘，这个算子本质上和vecadd是一样的。

- 给定两个形状相同的张量 A 和 B，维度为 [d1, d2, ..., dn]
- 输出张量 C 与输入张量形状相同，每个元素计算为： $C[i1, i2, \dots, in] = A[i1, i2, \dots, in] \cdot B[i1, i2, \dots, in]$

SiLU算子的实现

根据上面的叙述，我们将带领大家动手实现Triton版本的SiLU算子，我们先写它的调用函数：

代码块

```
1 def silu(x: torch.Tensor):
2     assert x.is_cuda, "Input tensor must be on CUDA device"
3     output = torch.empty_like(x)
4     n_elements = x.numel()
5     BLOCK_SIZE = 1024
6     grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
7
8     silu_kernel[grid](x, output, n_elements, BLOCK_SIZE=BLOCK_SIZE)
9     return output
```

x 是输入张量，将作为激活函数的输入；output 是用于存储激活函数计算结果的输出张量。在 Kernel 的实现中，我们采用大小为 1024 的 block 作为基本计算单位。

具体来说，输入张量会被划分为多个 block，每个 block 包含 1024 个元素。例如，当输入张量包含 5120 个元素时，它将被划分为 5 个 block 进行并行处理，也就是说我们网格中的block数量调整为5。

代码块

```
1 输入张量大小 N = 5120, BLOCK_SIZE = 1024 的情况:
2
3 |<-1024->|<-1024->|<-1024->|<-1024->|<-1024->
4 [Block 0][Block 1][Block 2][Block 3][Block 4]
```

接下来，我们深入分析silu_kernel的实现。根据前面的分析，每个kernel调用将处理一个大小为1024的block数据。根据SiLU公式： $x \cdot \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}$ ，我们首先需要从全局显存加载数据，通过 `tl.load(x_ptr + offsets, mask=mask)` 实现。在加载之前，我们需要确定当前kernel调用的数据处理范围，即从 $\text{pid} \times \text{BLOCK_SIZE}$ 开始，到 $\text{pid} \times \text{BLOCK_SIZE} + \text{BLOCK_SIZE}$ 结束。

随后，我们需要计算e的负指数幂。这里需要注意的是，为了保证运算的精度，需要先将输入数据转换为fp32类型 `to(tl.float32)`。接下来的流程是计算sigmoid值，并将其与输入相乘得到最终结果。由于此时结果仍然是fp32类型，我们需要将其转换回fp16类型，并通过 `tl.store` 将结果写回输出张量。至此，我们就实现了SiLU算子。

代码块

```
1 @triton.jit
```

```

2  def silu_kernel(
3      x_ptr,  # 输入张量的指针
4      output_ptr,  # 输出张量的指针
5      n_elements,  # 张量的总元素数
6      BLOCK_SIZE: tl.constexpr,  # 每个线程块处理的元素数
7  ):
8      # pid∈[0,5)
9      pid = tl.program_id(axis=0)
10     block_start = pid * BLOCK_SIZE
11     # 确定当前次调用中的数据加载范围 1x1024 + [0...1024]
12     offsets = block_start + tl.arange(0, BLOCK_SIZE)
13     mask = offsets < n_elements
14     # 加载fp16的输入数据
15     x = tl.load(x_ptr + offsets, mask=mask)
16     # 将输入类型转换为fp32的
17     x_fp32 = x.to(tl.float32)
18     # 计算sigmoid
19     sigmoid_x_fp32 = 1 / (1 + tl.exp(-x_fp32))  # sigmoid(x)
20     sigmoid_x = sigmoid_x_fp32.to(tl.float16)
21     # 转换类型到fp16
22     output = x * sigmoid_x.to(tl.float16)
23     # 将fp16类型的结果写回
24     tl.store(output_ptr + offsets, output, mask=mask)

```

对SiLU算子的实操

在这里我们将使用triton kernel来计算silu函数，输入的元素个数一共有n_elements个，也就是说一共有n_elements/BLOCK_SIZE个块，每个块处理会处理BLOCK_SIZE个元素。

代码块

```

1  def silu(x: torch.Tensor):
2      output = torch.empty_like(x)
3
4      n_elements = x.numel()
5      BLOCK_SIZE = 1024
6      grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']),)
7      silu_kernel[grid](x, output, n_elements, BLOCK_SIZE=BLOCK_SIZE)
8
9      return output
10
11 if __name__ == "__main__":
12     x = torch.randn(10000, device='cuda').half()
13     output_triton = silu(x)
14     output_torch = torch.nn.functional.silu(x)
15     print("最大差异:", torch.max(torch.abs(output_triton - output_torch)))

```

在这里我们将调用triton kernel——silu来计算silu函数，输入的元素个数一共有n_elements个，也就是说一共有n_elements / BLOCK_SIZE个线程块来进行并行计算，每个块处理会处理BLOCK_SIZE个元素。

MUL算子的实现

我们在前文中说过，这是一个将张量中的元素进行逐点相乘的操作，它的实现和VecAdd主体上是相同的，两个算子不同的只有两个输入元素之间的操作符。

在Mul算子中，同样地，我们需要从两个输入张量中分别加载i1和i2，加载的范围同样有当前kernel的id来确定，每个kernel的调用中，我们都会从两个输入张量中加载一个block大小的数据，然后我们在块内进行逐元素的乘法运算，最后将结果写回到输出张量中。以下是它的实现：


```

1  @triton.jit
2  def vector_mul_kernel(
3      x_ptr,  # 第一个输入张量的指针
4      y_ptr,  # 第二个输入张量的指针
5      output_ptr,  # 输出张量的指针
6      n_elements,  # 张量的总元素数
7      BLOCK_SIZE: tl.constexpr,  # 每个block处理的元素数
8  ):
9      # 获取当前程序的ID
10     pid = tl.program_id(axis=0)
11
12     # 计算当前block的起始位置和偏移量
13     block_start = pid * BLOCK_SIZE
14     offsets = block_start + tl.arange(0, BLOCK_SIZE)
15
16     # 创建有效数据的掩码
17     mask = offsets < n_elements
18
19     # 从全局内存加载数据
20     x = tl.load(x_ptr + offsets, mask=mask)
21     y = tl.load(y_ptr + offsets, mask=mask)
22
23     # 执行逐元素乘法
24     output = x * y
25
26     # 将结果写回全局内存
27     tl.store(output_ptr + offsets, output, mask=mask)

```

我们用offsets确定当前kernel调用中需要处理的数据范围，然后通过tl.load从全局显存中加载数据，两个输入分别记作x和y，并使用mask来确保我们只处理有效的数据。随后，我们进行逐元素的乘法运算，并调用tl.store将结果写回到输出张量中。

MLP算子的Triton实现（A@B）

我们已经实现了SiLU激活函数和MUL算子，接下来就可以开始实现MLP算子了。从pytorch实现的代码中可以看出，MLP算子由三个线性层组成，分别是w1,w2,w3。另外还有激活函数，SiLU以及一个MUL运算组成。

在此之前，我还想说，在Pytorch中，nn.Linear算子的运算过程为 $x @ w.t() + b$ ，其中x为输入，w为权重，b为偏置。所以如果我们需要自己用Triton实现算子，我们需要先将权重的转置保存，随后再进行计算。

代码块

```

1  def mlp(inputs, w1t, w2t, w3t):
2      w1x = matmul(inputs, w1t, True)
3
4      w3x = matmul(inputs, w3t)
5      mul_out = vector_mul(w1x, w3x)
6      out = matmul(mul_out, w2t)
7      return out

```

1. 对照Pytorch的mlp层实现，我们先调用matmul算子计算输入inputs和权重w1t的矩阵乘法，得到w1x。
2. 随后，我们再计算w3x，这里我们注意到，w3x的计算中，输入是inputs，权重是w3t，所以和Pytorch的实现是一致的。
3. 随后再将w1x和w3x传入vector_mul算子，得到mul_out，最后将mul_out和w2t传入matmul算子，得到最终的输出out。

至此就完成了MLP层的计算流程。

结果的比较

Mlp层的定义

Linear

```
CLASS torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None) [SOURCE]
```

Applies an affine linear transformation to the incoming data: $y = xA^T + b$.

为了验证Triton实现的准确性，我们进行了5次循环测试。在每次迭代中，我们分别调用自定义的mlp函数和PyTorch原生的mlp层，并记录它们的执行时间。通过计算两者输出结果的绝对差值平均值，我们能够量化实现之间的差异。实验结果表明，两者的输出差异稳定在1e-5量级，这一微小的误差在浮点运算的可接受范围内，充分证明了我们的Triton实现与PyTorch原生实现具有高度一致性，验证了实现的正确性。

结果验证

代码块

```
1  triton_time = 0
2  torch_time = 0
3  for i in range(5):
4      t1 = time.time()
5      output2 = mlp(inputs, w1t, w2t, w3t)
6      t2 = time.time()
7      triton_time += t2 - t1
8
9      t1 = time.time()
10     outputs = module(inputs)
11     t2 = time.time()
12     torch_time += t2 - t1
13     print(torch.mean(torch.abs(output2 - outputs)))
14
15     print('triton:{}'.format(triton_time / 5))
16     print('torch:{}'.format(torch_time / 5))
17     print('-')
18
```

补充实现A@B.t()

A@W.T的实现

在上面的例子中，我们需要先将w进行转置，随后再传入到mlp函数中，这是因为我们实现的是A@W，而不是A@W.T，所以本章将带大家实现一下A@W.T的Triton算子。其实这个算子在上一节的fp8_gemm8中已经实现过了，这里我们再实现一遍，加深大家的理解。

1. 首先是加载m和n轴上的分块以及k轴上的分块

代码块

```
1  pid_m = tl.program_id(axis=0)
2  pid_n = tl.program_id(axis=1)
```

2. 根据m和n轴上的分块确定当前对应的偏移量，分别是offs_m和offs_n；

```

1 offs_m = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
2 offs_n = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
3 offs_k = tl.arange(0, BLOCK_SIZE_K)

```

3. 根据k轴上的分块确定当前对应的偏移量，分别是offs_k；

4. 通过偏移量，确定矩阵A和矩阵W的偏移位置，从二维分析当前块的坐标就是(offs_m, offs_k)和(offs_k, offs_n)；

代码块

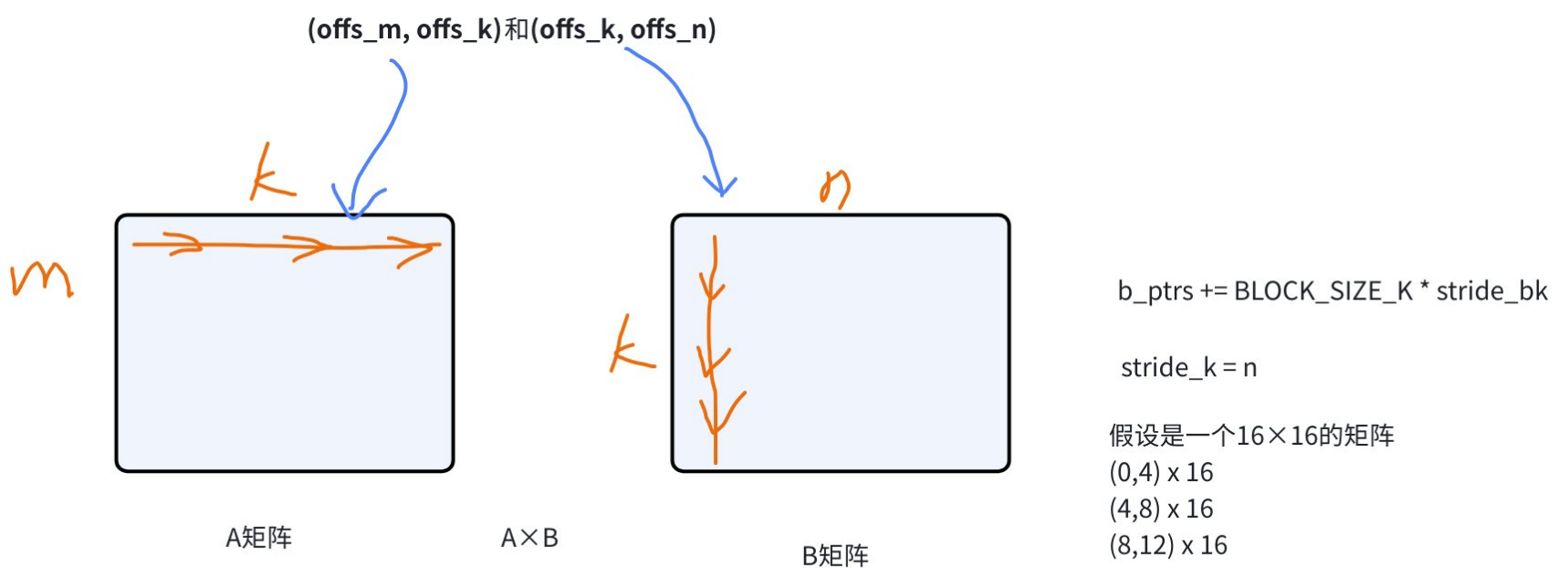
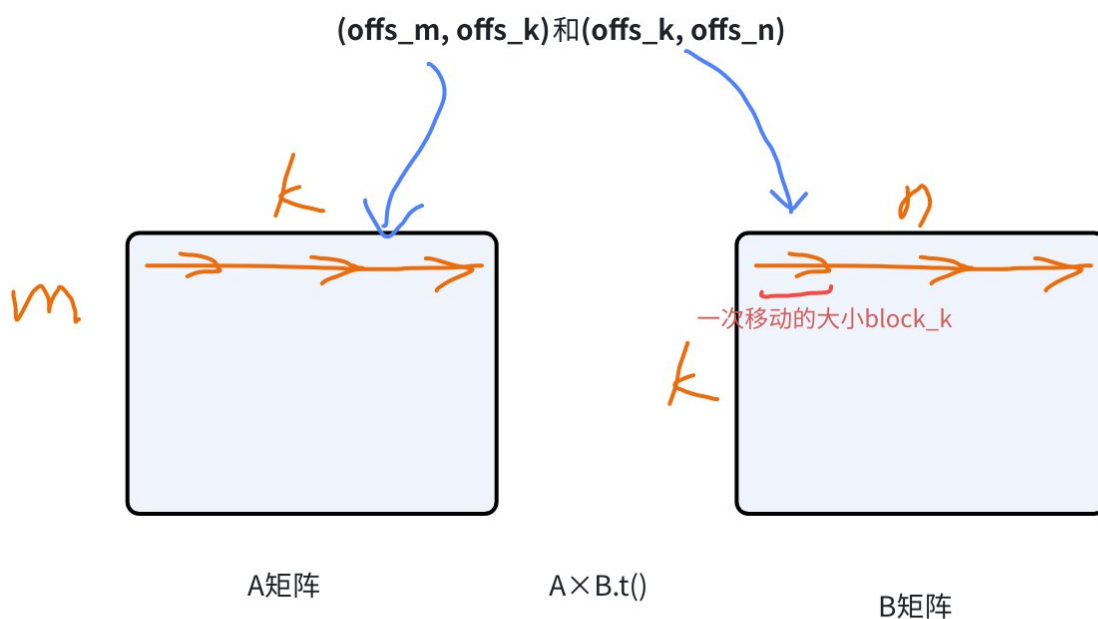
```

1 a_ptrs = a_ptr + offs_m[:, None] * K + offs_k[None, :]
2 b_ptrs = b_ptr + offs_n[None, :] * K + offs_k[:, None]

```

5. 随后我们要做的就是规约轴上进行规约，`a_ptrs += BLOCK_SIZE_K` 就是按行在a某行的不同列中移动，所以对于a来说它的规约轴就是K轴；B矩阵同理，在同一行的不同列上进行移动，它的规约轴是N轴。这里没看明白的话可以回过头看一下deepseek gemm fp8量化那一节。

从图中也可以看出，对矩阵B的归约轴不同会导致对B的数据读取计算方式和先后的不同，分别可以完成A@B.t()和A@B的计算。



代码块

```

1 @triton.jit
2 def fp32_gemm_kernel(a_ptr, b_ptr, c_ptr,
3                       M, N: tl.constexpr, K: tl.constexpr,
4                       BLOCK_SIZE_M: tl.constexpr,
5                       BLOCK_SIZE_N: tl.constexpr,

```

```

6         BLOCK_SIZE_K: tl.constexpr):
7     pid_m = tl.program_id(axis=0)
8     pid_n = tl.program_id(axis=1)
9     k = tl.cdiv(K, BLOCK_SIZE_K)
10    offs_m = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
11    offs_n = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
12    offs_k = tl.arange(0, BLOCK_SIZE_K)
13    # 从二维分析当前块的坐标就是(offs_m, offs_k)和(offs_k, offs_n);
14    a_ptrs = a_ptr + offs_m[:, None] * K + offs_k[None, :]
15    b_ptrs = b_ptr + offs_n[None, :] * K + offs_k[:, None]
16
17    accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
18    for i in range(k):
19        a = tl.load(a_ptrs, mask=offs_k[None, :] < K - i * BLOCK_SIZE_K, other=0.0)
20        b = tl.load(b_ptrs, mask=offs_k[:, None] < K - i * BLOCK_SIZE_K, other=0.0)
21        accumulator += tl.dot(a, b)
22        # 如上图那样移动
23        a_ptrs += BLOCK_SIZE_K
24        b_ptrs += BLOCK_SIZE_K
25    c = accumulator.to(c_ptr.dtype.element_ty)
26    offs_m = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
27    offs_n = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
28    c_ptrs = c_ptr + offs_m[:, None] * N + offs_n[None, :]
29    mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
30    tl.store(c_ptrs, c, mask=mask)

```

调用样例

```

代码块
1  def call_fp32_gemm():
2      # 定义矩阵维度
3      M, N, K = 128, 128, 128
4      BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K = 32, 32, 32
5
6      # 创建随机fp32矩阵和缩放因子
7      a = torch.randn(M, K, dtype=torch.float32, device='cuda')
8      b = torch.randn(N, K, dtype=torch.float32, device='cuda')
9      c = torch.zeros((M, N), dtype=torch.float32, device='cuda')
10
11     # 计算网格大小
12     grid = lambda META: (triton.cdiv(M, META['BLOCK_SIZE_M']), triton.cdiv(N, META['BLOCK_SIZE_N']))
13
14     # 调用kernel
15     fp32_gemm_kernel[grid](a, b, c, M, N, K, BLOCK_SIZE_M, BLOCK_SIZE_N, BLOCK_SIZE_K)
16     print('mean:{}'.format(torch.mean(torch.abs(a@b.t() - c))))
17     return c
18
19 if __name__ == '__main__':
20     call_fp32_gemm()

```

这里需要注意的是矩阵B的大小是(N, K)，矩阵A的大小是(M, K)，所以矩阵b的转置大小是(K, N)，矩阵A和矩阵B的转置相乘得到矩阵c，矩阵C的大小是(M, N)。